

More Parking System Classes: Write Up

ICT 4305: Object Oriented Methods and Programming

for

Master of Science

Information Technology

Kalika Browder

University of Denver College of Professional Studies

October 19, 2025

Faculty: Nirav Shah, M.S. , MBA

Director: Cathie Wilson, M.S.

Dean: Bobbie Kite, PhD

Reflection

To start this assignment, which focused on modeling a small parking system and further implementing the core services required by the parking office (and also built off of my previous assignments), I had to take careful and intricate work to determine if I was both doing things correctly, as well as ensuring that the system didn't have bugs. The most straightforward part for me was capturing the main domain concepts. These include Customer, Car, Permit, ParkingLot, ParkingCharge, and Money. The more challenging parts of this whole process were deciding the exact responsibilities for each class and handling the two different scanning behaviors (ENTRY_ONLY vs ENTRY_EXIT) in a way that was understandable, simple, and expandable for future use.

To manage the complexity of the assignment, I leaned on two complementary strategies that came from our learning materials within the past couple of weeks. The first was noun-phrase extraction from the use cases to enumerate candidate classes, while the second was to use responsibility assignment to limit each class to a single and clear role. This helped keep the domain model clean and made the Java classes more intuitive for me to implement. I also used "Money" as a value object with "long cents" to avoid point rounding errors, and to overall simplify the entire process. This small design decision prevented subtle bugs in monetary calculations, which I wanted to avoid completely.

One implementation decision that was thought over heavily was making ParkingLot responsible for detecting both entries and exits and for calculating charges in ENTRY_EXIT mode, while the ParkingOffice was to centrally store all of the ParkingCharge records. This separation keeps a lot of specific rules encapsulated in ParkingLot while maintaining a single

source of truth for charges and customers in the ParkingOffice. I also intentionally used Java `List<T>` and `Optional` to illustrate Generics (and previous weeks learning materials) and modern collection patterns.

To further extend my work this week, I also wanted to take the discount step out of the way, so I added an automatic discount calculation (20% for compact cars) directly within the system's logic. This enhancement made the Parking System more realistic in terms of the assignment's description, and also allowed for the discount to now be applied automatically whenever a compact car enters or exits any University lot. Incorporating the feature also gave me a better understanding of how to integrate business rules into code without disrupting other components, because initially, I didn't include the discount. However, it was rewarding to see how a small and targeted change could in general make the overall design more functional and aligned with the original problem requirements. This assignment also reinforced that fateful upfront design (even at the domain model levels, etc.) makes implementation in code more straightforward for designers and avoids rework later. The sequence diagrams and class model was invaluable for me this week in validating flows and writing code.